

ФГБОУ ВПО КубГУ

Курс лекций

2026 г

Технологии программирования
5 сем. Язык JAVA

Зав. каф. ВМиИ, к.ф.-м.н,
доцент, Фоменко С.И.

Тема 2:
ООП в Java. Инкапсуляция

Концепция ООП

Фундаментальные концепции:

1. **Инкапсуляция;**
2. **Наследование;**
3. **Полиморфизм;**





Инкапсуляция

Инкапсуляция -- объединение атрибутов и методов в одной сущности (объекте).

Объект – это черный ящик, внешняя среда не знает полностью, как устроен объект внутри. Взаимодействие с объектом осуществляется посредством интерфейсов (публичных методов и свойств).

Ограничение доступа к определенным атрибутам и/или методам называется **приватностью (скрытием)данных**. Скрытие данных является основной частью инкапсуляции.

При хорошем объектно-ориентированном проектировании объект должен показывать только интерфейсы, необходимые другим объектам для взаимодействия с ним. Детали, не относящиеся к задачи взаимодействия с объектом, должны быть скрыты от всех других объектов.

Инкапсуляция



https://smartiqa.ru/courses/python/lesson-6

Объявление класса (базового)

```
class Имя_Класса
{
    тело_класса //-> Инкапсуляция: данные, методы и способы
                    доступа к ним
}
```

`тело_класса:`

1. `Объявление членов класса (свойства-поля данных)`
2. `Объявление методов (функций-членов)`

Объявление класса (базового)

```
class Имя_Класса
{
    тело_класса
};
```

тело_класса:

1

```
Объявление полей (атрибутов) класса:
  [модификатор] тип имя_поля1,…;
  [модификатор] тип имя_поля2 = значение_по_умолчан;
```

Пример:

```
class Point
{
    double x=0,y=0; // атрибуты x и y
}
```



1. Объявление класса

модификатор поля/метода:

Модификаторы доступа:
<пусто> – (по умолчанию) члены класса доступны только внутри данного пакета (package-private)
public – открытый член класса: доступ везде
private – закрытый член класса: доступа извне нет
protected – защищённый член класса: доступ наследникам

Модификаторы свойств

static –принадлежность к классу, а не к экземпляру

final – финализируемый (неизменный) член класс
abstract – абстрактный члена класса/класс

Есть еще, рассмотрим позже.



1. Объявление класса

модификатор поля/метода:

```
class Const
{
    public static final double pi =3.14159926; //
открытый, неизменный статичный параметр == константа
    public static final double e = 2.7182818;
}

 ….

int main()
{
   double ang30=30*Const.pi/180
}
```

2. Объявление класса: методы

2

Объявление_методов

```
[модификатор] выходной_тип имя_метода(входные_параметры)
{
 тело функции
}
```

выходной_тип: стандартные тип данных, производный класс или void
Если void (пустой тип), то это процедура фактически

входные_параметры:
1) Фиксированный список формальных параметров функции

входные_параметры=[описатель_параметра1 [, описатель_параметра2 [,…]]]
описатель_параметра = тип имя_параметра

входные_параметры:
2) Переменный список входных параметров

описатель_параметра = тип ... имя_массива_параметров

В Java все параметры функций передаются по значению!



2. Объявление класса: методы

```
static public double sum2Forces(double force1, force2) {
    return force1+force2
 }

….
System.out.println("Суммарная сила: " + sum2Forces(1.0,2.0));
```

```
static public double sumForces(double... forces) {
    double sum = 0;
    for (double f : forces) {
        sum += f;
    }
     return sum
}
…
 double s0 =  sumForces();
 double s1 =  sumForces(1.0);
 double s2 =  sumForces(1.0,2.0);
 double s3 =  sumForces(1.0,2.0,3.0);
```



2. Объявление класса: методы с переменными параметрами

Правила с переменными параметрами

1) Varargs-параметр должен быть последним в списке параметров:

```
void applyLoad1(String type, double... values) { ... }      // OK
void applyLoad2(double... values, String type) { ... }      // ошибка компиляции
```

2) Допустим только один varargs-параметр на метод

```
void bad(double... a, int... b) { ... } // ошибка
```

3) Внутри метода varargs -- это обычный массив:

```
{   int n = forces.length;
    double first = forces[0];}
```

4) Можно передать массив в функцию явно:

```
double[] arr = {10.0, 20.5};
sumForces(arr); // работает точно так же
```

Вероятностный подсчёт

Как хорошо известно, можно держать пари, что в компании из 25 случайно выбранных людей есть по крайней мере двое с совпадающим днём рождения. Хотя на первый взгляд такое пари может показаться глупым, вероятность его выигрыша составляет 56%.

Ответ: Вероятность $1 - 365!/(340! * 365^{25})$.

Задача проверить.

Для получения псевдо-случайного числа можно воспользоваться функциями

1) Math.random() -- возвращает случайное число 0..1

2) import java.util.Random;

```
Random rand = new Random();            // seed по умолчанию (на основе времени)
// Random rand = new Random(42);
int n = rand.nextInt(N);               //  целое число из [0, N)
double r = rand.nextDouble(); //  r in [0.0, 1.0)
```

2. Экземпляр класса

Класс – это тип данных, экземпляр класса – это конкретный объект этого типа.
Все объекты – ссылки



Объявление новой переменной как экземпляра класса

```
Имя_класса Имя_переменной
```

Динамическое размещение (new) экземпляра из памяти

```
Имя_указателя = new Имя_класса;   или
Имя_указателя = new Имя_класса(параметры);
```

Доступ к свойствам и методам экземпляра

```
Имя_переменной.свойство_или_метод
```

Пример:

```
class Point{…}
  ..
int main()
{
    Point myPoint=new Point;
    myPoint.x = 1; myPoint.y = 2;
    double r = myPoint.rad();
}
```

Служебный формальный параметр this

this – это ссылка на экземпляр класса, для которого был вызван метод

```
выходной_тип имя_метода(входные_параметры)
{
   … this.свойство_или_метод …
}
```

Пример:

```
class Point
{
  double x,y;
   double rad()
   {
     return Math.sqrt(this.x * this.x + this.y * this.y);
   }
};
```

3. Конструкторы

Конструктор– это особый метод класса, который вызывается при создании экземпляра класса.



```
class Имя_Класса
{
    …
    Имя_Класса (входные_параметры) //конструктор
};
```

```
class Point
{
     public double x,y;
     public Point(double x, double y)
     {
      this.x=x;
      this.y=y;
     }
};
```



4. Сеттер и геттер

Сеттер (setter) – метод для установки значения свойства

Геттер (getter) – метод для получения значения свойства

```
class Point
{
  private double x,y; //приватные члены
  public Point (x0,y0){x=x0; y=y0;} // конструктор

  public  void set_x(double x) { this.x=x;} // сеттер x
  public double get_x() { return this.x;} // геттер x
};

…

Point p=new Point(0,1);
// double x=p.x; // недоступно
double x=p.get_x(); // доступно
```



5. Перегрузка

Перегруженные функции/методы – это функции с одинаковым именем, но разным набором параметров (разная сигнатура)

```
выходной_тип_1 имя_функции (входные_параметры_1)
{
    тело_1
}

выходной_тип_2 имя_функции (входные_параметры_2)
{
    тело_2
}
```

Формальные параметры (типы данных или их очередность) или выходной тип данных должны отличаться, чтобы компилятор при трансляции мог отличить разные вызовы.



5. Перегрузка методов

Пример:

```
 class Point
 {
 private  double x,y;
        …
 public  double dist() // расстояние до центра коорд.
 {
  return Math.sqrt(x*x+y*y);
 }

 public double dist(Point b) // расстояние до точки b.
 {
   return Math.sqrt((x-b.X)*(x-b.X)+(y-b.y)*(y-b.y));
 }
 };
...
Point P1=new Point(1, 2);
Point P2=new Point(0, 1);
System.out.printf("|P1|=%f\n",P1.dist());
System.out.printf("|P2-P1|=%f\n",P2.dist(P1));
```



5. Перегрузка конструктора

Пример 1 :

```
 class Point
 {
   public double x,y;

   public Point(double x0, double y0){ x=x0; y=y0;}
   public Point(double xy0){x=xy0; y=xy0;}
   public Point(Point source) // конструктор копий
    {
          this.x=source.x;
          this.y=source.y;
    }
 ...
}
...

Point P,P0;
P = new Point(1,2);
P0 = new Point(0);
Point P2 = new Point(P);
```



5. Перегрузка конструктора

Пример 2 (делегирующий конструктор):

```
 class Point
 {
   public double x,y;

   public Point(double x0, double y0){ x=x0; y=y0;}
   public Point(double x0)
  {
        //x=y=x0;
        this(x0,x0); //вызов конструктора Point(x0,x0)
  }
 public Point()
  {
        this(0,0); //вызов конструктора Point(0,0)
  }     ...
}
...

Point P,P0;

P=new Point(1,2);
P0=new Point(0);
```

6. Анонимные объекты

Вызывать методы можно непосредственно после создания объекта в той же строке



Пример

```
class Point
{
 private  double x,y;
 public Point(double x0, double y0){ x=x0; y=y0;}
 public Point(double x0){this(x0,x0);}

 public double dist(Point b) // расстояние до точки b.
 {
   return Math.sqrt((x-b.x)*(x-b.x)+(y-b.y)*(y-b.y));
 }
 ...
};
...
double d = new Point(1,2).dist(new Point(1));
System.out.printf("|P2-P1|=%f\n", new Point(2,1).dist(new Point(2)));
```



Литература

1. Г. Шилдт, Java. Полное руководство, 12-е изд. : Пер. с англ. - СПб. "Диалектика"; 202